Grokking, Reverse-Engineered: A Mechanistic Account of Delayed Generalisation on Modular Arithmetic

Punnag Choudhury

August 2026

Abstract

A one-layer transformer trained on modular addition `(a + b) mod p` exhibits *grokking*: it fits the training set almost immediately but does not generalise until tens of thousands of optimisation steps later. We reproduce this phenomenon from scratch in PyTorch and reverse-engineer the trained network. In the discrete Fourier basis over `Z_p` the grokked token embedding is sparse — 96% of its power sits on three frequencies, with the Gini sparsity of the power spectrum rising `0.05` to `0.93` across the transition. The network's answer-determining signal is a closed-form trigonometric expression, a sum of cosines `sum_k cos(w_k (a + b - c))`, recovered to `R^2 = 0.9999` and peaking exactly at `c = a + b`; reduced to nothing but that formula the network still classifies at 100%, though the formula accounts for only ~66% of the raw logit variance, a caveat we report rather than hide. We verify the mechanism causally by editing the embedding in Fourier space: the key frequencies are necessary, sufficient, and specific. Replaying the training trajectory, progress measures show the circuit becomes the dominant mechanism ~2,200 steps *before* generalisation is visible in the test loss — the "sudden" jump is the tip of a gradual, three-phase process (memorisation, circuit formation, cleanup). Grokking requires weight decay (with none, the model memorises forever and never groks) and a minimum data fraction; every seed learns the *same* algorithm on *different* frequencies; and, sweeping the modulus at fixed model capacity, the effective number of learned frequencies stays roughly constant while `p` grows several-fold — the circuit size is set by capacity, not by the problem. This work reproduces and consolidates the mechanistic account of Nanda et al. (2023); the analysis primitives are unit-tested on synthetic inputs and every figure is regenerable from a clean checkout.

Introduction and motivation

Grokking (Power et al., 2022) is a striking failure of the intuition that generalisation tracks training loss. On small algorithmic tasks a network can reach perfect training accuracy while its test accuracy remains at chance for a very long time, and then — with no change in the training signal, and long after the training loss has flatlined — generalise abruptly. It is a clean counterexample to the folk model in which memorisation and generalisation are opposite ends of a single overfitting axis.

Grokking is also a rare gift to interpretability. Neural networks are usually opaque because we do not know the function they are meant to compute, so "what is the model doing?" has no reference answer. Here the ground truth is *known* — the task is modular arithmetic — which makes every mechanistic claim falsifiable by intervention: name the mechanism the network is supposed to use, delete it, and the accuracy must collapse. This report reproduces grokking on modular addition and reverse-engineers the resulting network, following the mechanistic-interpretability programme of Nanda et al. (2023) and building on the "clock and pizza" analysis of Zhong et al. (2023).

Our contribution is a careful, from-scratch reproduction and a verified reverse-engineering of the grokking circuit, packaged so that every claim is checkable. Concretely:

1. A from-scratch, fully hookable one-layer transformer (named weight matrices, explicit einsums, an activation cache, no LayerNorm) and a seeded, bit-reproducible full-batch training pipeline that reproduces grokking across three seeds.
2. A Fourier-basis analysis over $\mathbb{Z}_p$ showing the grokked embedding is sparse (Gini $0.05 \to 0.93$) and identifying the handful of key frequencies it uses.
3. The clock circuit recovered symbolically: the decision function is $\sum_k \cos(\omega_k(a+b-c))$ to $R^2 = 0.9999$ and classifies at 100%, reported honestly alongside the $\sim$66% raw-logit-variance caveat.
4. A causal verification — necessity, sufficiency and specificity — by editing the embedding in Fourier space and re-running the otherwise-untouched network.
5. Progress measures computed along the trajectory that reveal circuit formation $\sim$2,200 steps before the test loss moves, resolving grokking into three phases.
6. Ablations mapping what grokking depends on: weight decay (essential), a train-fraction threshold, and the operation.
7. A cross-seed result: every seed learns the clock on a *different* frequency set — a universal algorithm with seed-specific parameters.
8. A scaling result: at fixed model capacity the effective number of learned frequencies is roughly constant as the modulus p grows several-fold.
9. Tested analysis primitives — the Fourier and circuit code is unit-tested on a synthetic clock with known answer — and full reproducibility.

Background

Grokking

Following Power et al. (2022), grokking is studied on small algorithmic datasets where a fixed fraction of all input pairs is used for training. A network trained with weight decay first *memorises* — reaching perfect training accuracy while generalising no better than chance — and then, after a long plateau, generalises. The delay between memorisation and generalisation can be tens of thousands of steps, and the transition in test accuracy is abrupt.

The task

The task is $(a \operatorname{op} b) \bmod p$ for a prime modulus p, with the operation one of addition, subtraction, or multiplication (addition unless stated). Each ordered pair (a, b) with $0 \le a, b < p$ is one example, presented as the three-token sequence $[a, b, =]$, where the "=" token has index p; the answer is read off the final position over the p output classes. A fixed random fraction (30% unless stated) of the p^2 pairs forms the training set and the remainder is held out, so generalisation means inferring the group operation on pairs never seen in training.

The model

The model is a one-layer transformer with residual-stream width $d = 128$, four attention heads of dimension 32, and an MLP hidden width of 512. Writing W_E for the token embedding and W_{pos}

for the learned positional embedding, the forward pass on a token sequence is

$$\begin{aligned}\text{resid}_{\text{pre}} &= W_E[\text{tokens}] + W_{\text{pos}},\\ \text{resid}_{\text{mid}} &= \text{resid}_{\text{pre}} + \text{Attn}(\text{resid}_{\text{pre}}),\\ \text{resid}_{\text{post}} &= \text{resid}_{\text{mid}} + \text{MLP}(\text{resid}_{\text{mid}}),\\ \text{logits} &= \text{resid}_{\text{post}}[-1]\, W_U,\end{aligned}$$

with attention using per-head projections W_Q, W_K, W_V, W_O and a causal mask, and the MLP a two-layer network with a ReLU nonlinearity. There is no LayerNorm and there are no biases, so the trained network is a small, legible set of linear maps around one attention block and one MLP — the configuration in which the grokking circuit has a clean closed form. Training is in float32, since grokking is a statement about optimisation dynamics rather than high-precision numerics.

The Fourier basis over $\mathbb{Z}_p$

The task lives on the cyclic group $\mathbb{Z}_p$, so the natural basis for any quantity indexed by a number token is the discrete Fourier basis. For p odd we use the orthonormal basis F of $\mathbb{R}^p$ whose rows are the constant $F_0(n) = 1/\sqrt{p}$ and, for each frequency $k = 1, \dots, (p-1)/2$,

$$F_{2k-1}(n) = \sqrt{\tfrac{2}{p}}\,\cos\!\Big(\tfrac{2\pi kn}{p}\Big), \qquad F_{2k}(n) = \sqrt{\tfrac{2}{p}}\,\sin\!\Big(\tfrac{2\pi kn}{p}\Big).$$

Because F is orthonormal ($FF^\top = I$), projecting a weight matrix onto it redistributes but never inflates its total power (Parseval), so "the fraction of power at frequency k" and the Gini sparsity of the power spectrum are both well-defined. Every mechanistic result below is expressed in this basis.

Method

Reverse-engineering in the Fourier basis

We analyse a trained model by projecting the number-token rows of its embedding, $W_E[0{:}p]$ (shape $p \times d$), onto the Fourier basis and summing the cos/sin power at each frequency k. The *power spectrum* is that per-frequency power; its concentration is summarised by the Gini coefficient (0 for a uniform spectrum, $\to 1$ for one concentrated on a single frequency), and the *key frequencies* are the minimal set carrying 90% of the non-constant power.

The decision function and the clock fit

To read the circuit off the logits we form the *decision function*. Let $L[a, b, c]$ be the logit for answer c on input (a, b), mean-centred over c (the softmax, and therefore the prediction, is invariant to a constant added to a logit vector). The decision function averages L over all triples with a fixed $d = (a + b - c) \bmod p$,

$$f(d) = \text{mean}\,\{\, L[a,b,c] \;:\; (a+b-c) \equiv d \pmod p \,\}.$$

The clock hypothesis is that f is a sum of cosines at the key frequencies, $f(d) = \sum_k A_k \cos(\omega_k d + \varphi_k)$ with $\omega_k = 2\pi k/p$, maximised at $d = 0$ (i.e. $c = a + b$) because every cosine interferes constructively there. We fit $f(d)$ to $\sum_k [A_k \cos + B_k \sin](\omega_k d)$ by least squares and report the R^2 of the fit, the fraction of L's variance that f explains, and the accuracy of the pure classifier $\text{argmax}_c\, f(a+b-c)$.

Causal ablation by frequency editing

Because the circuit reads its frequencies out of the embedding, we intervene there. Given a set of frequencies, we project $W_E[0{:}p]$ onto (or off) those frequencies in the Fourier basis, write the edited embedding back into an otherwise-untouched copy of the model, and re-run it. Keeping only the key frequencies tests sufficiency; removing only them tests necessity; removing any other single frequency tests specificity.

Progress measures

To see when the circuit forms we replay the trajectory's saved weight snapshots. Fixing the *final* model's key frequencies as the target circuit, at each snapshot we compute the *restricted* loss (train loss with the embedding projected onto the key frequencies) and the *excluded* loss (with the key frequencies removed), along with the embedding sparsity and the weight norm. The step at which the restricted loss falls below the excluded loss — where keeping only the key frequencies fits the data better than removing them — marks the point at which the key-frequency circuit becomes the dominant mechanism.

Verification of the analysis

The interpretability code is unit-tested on a *synthetic* clock: a logit tensor built as $\sum_k \cos(\omega_k(a+b-c))$ with known frequencies and answer. The analysis must recover f to $R^2 = 1$, unit amplitudes, a peak at $d = 0$, 100% accuracy, and exactly the planted frequencies. This pins down that a headline number is a property of the trained model and not an artefact of the analysis pipeline.

Experimental setup

All runs use PyTorch 2.11 with pinned dependencies and fixed seeds; a CUDA GPU is optional. Training is full-batch AdamW (learning rate 10^{-3}, weight decay 1.0, betas $(0.9, 0.98)$) on the fixed training split; grokking runs use 40,000 steps and log-spaced weight snapshots are saved for the trajectory analysis. The canonical configuration is $p = 113$ with a 30% training fraction. The correctness anchors are the *known task* (every mechanistic claim is checked by intervention against the ground-truth modular-addition labels) and the *synthetic-clock* unit tests described above; every figure is a committed file under `results/` regenerable via `make reproduce`.

Results

Every figure is a committed file under `results/`; every number traces to a committed table.

Grokking reproduced

Grokking reproduces cleanly and robustly (Figure 1). Across three seeds the model reaches 100% training accuracy by step 200 while test accuracy remains at chance, then generalises at steps 4,800 / 6,000 / 8,400 — a mean delay of 6,200 steps between memorisation and generalisation. The phenomenon is robust; only its timing is seed-dependent.

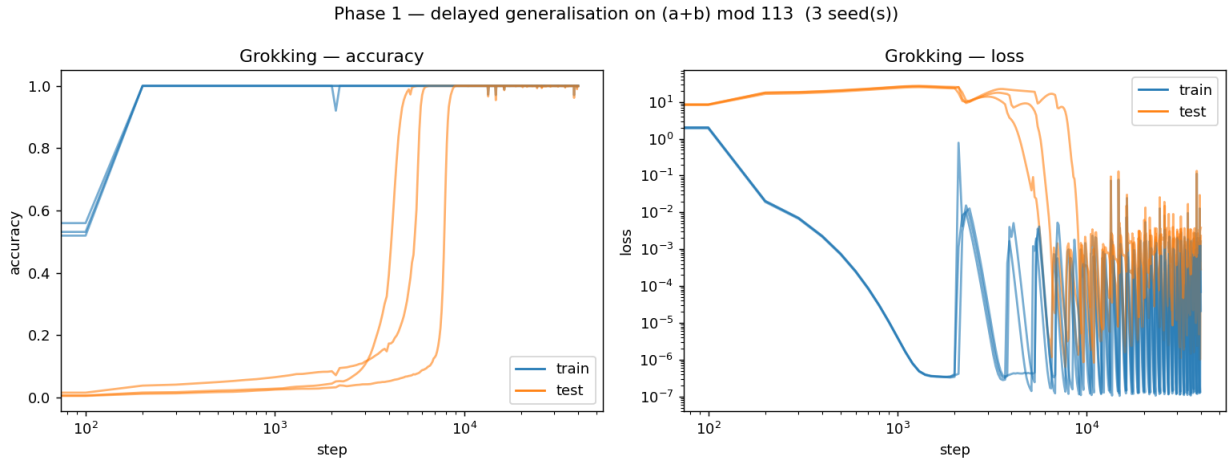

Figure 1: Across three seeds, training accuracy (blue) reaches 100% by step 200 while test accuracy (orange) stays at chance for thousands of steps before snapping to 100%.

The embedding is sparse in Fourier space

After grokking, the token embedding is sparse in the Fourier basis: 96% of its power sits on three frequencies ($k = 1, 8, 21$ for seed 0), whereas at initialisation the same power is spread diffusely across all 56 frequencies (Figure 2). The Gini coefficient of the power spectrum rises $0.05 \to 0.25 \to 0.93$ across initialisation, the memorisation plateau, and the grokked model. The network has learned to represent numbers using a handful of periodic features.

The circuit: a sum of cosines

The decision function $f(d)$ is a sum of cosines at the three key frequencies, recovered to $R^2 = 0.9999$, and it is maximised at $d = 0$ — i.e. at $c = a + b$ (Figure 3). Reduced to nothing but $\text{argmax}_c\, f(a + b - c)$, this formula classifies the task at 100%. We report the scope honestly: $f(d)$ accounts for $\sim 66\%$ of the raw centred-logit variance; the residual is c-structure not of the form $(a + b - c)$ that does not change the argmax, which is why the pure formula still classifies perfectly. This is the network's decision rule, expressed in closed form — the analogue, in a transformer, of reading a known formula back out of a trained model.

Causal verification

Editing the embedding in Fourier space and re-running the otherwise-untouched network confirms the circuit causally (Figure 4). Keeping *only* the three key frequencies retains 100% accuracy (sufficient); removing *only* those three collapses accuracy to chance (necessary); removing any other single frequency has no effect (specific). An honest detail: removing one of the three key frequencies alone drops accuracy only to $\sim 25\%$ — the other two still carry signal — and it takes removing all three to reach chance.

Progress measures: grokking is gradual

Grokking looks discontinuous, but the internal measures reveal a continuous process (Figure 5). The restricted and excluded losses cross over at step 2,791 — where the key-frequency circuit fits

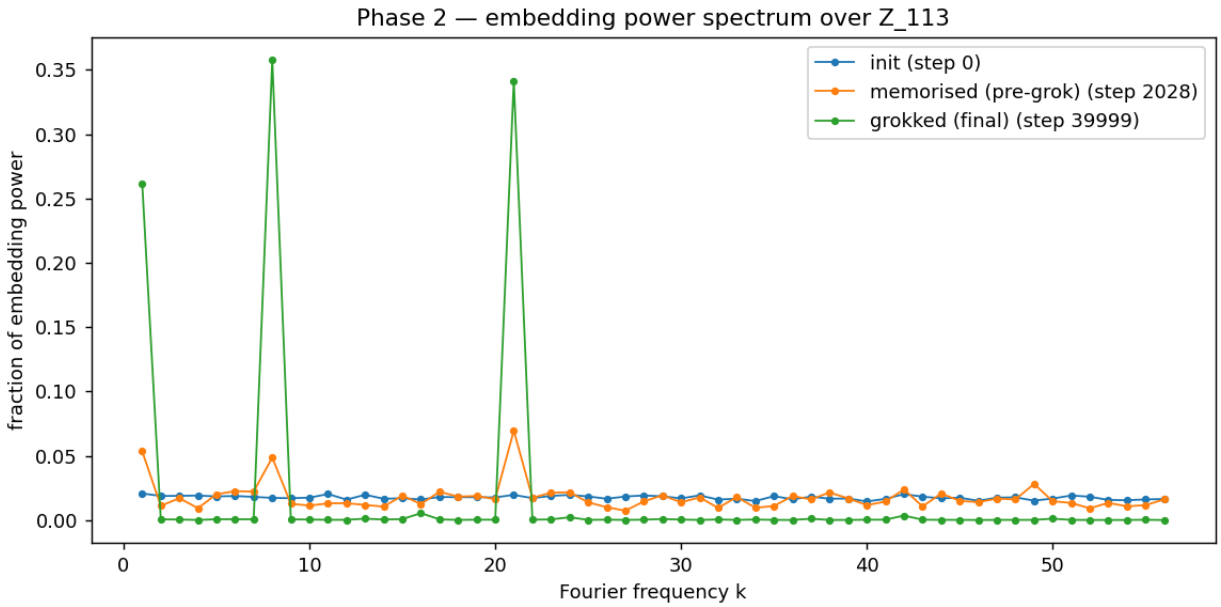

Figure 2: The grokked embedding (green) concentrates its power on three Fourier frequencies; the random-init (blue) and memorised-but-not-grokked (orange) embeddings are diffuse.

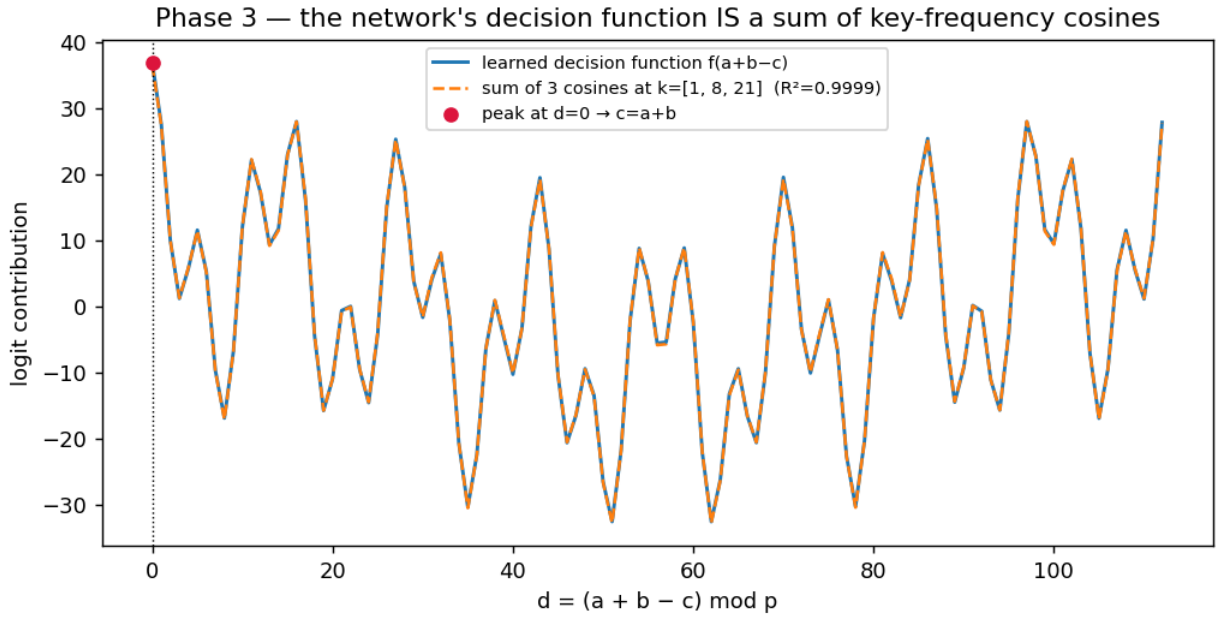

Figure 3: The learned decision function (blue) and a sum of three cosines (orange dashed) coincide at R-squared = 0.9999, peaking at d = 0 (c = a + b).

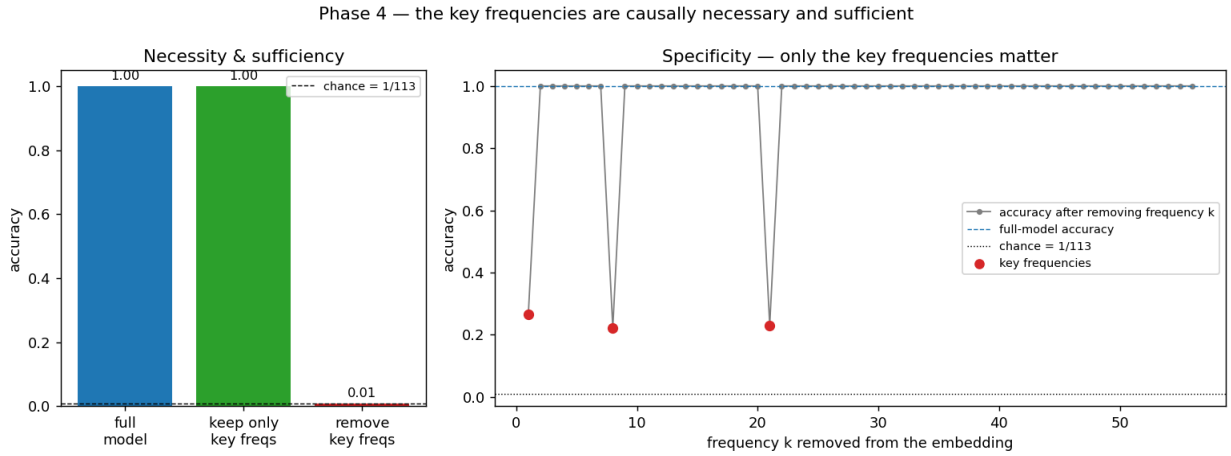

Figure 4: Keeping only the three key frequencies preserves 100% accuracy; removing them drops to chance; removing any other frequency does nothing.

the training data better than everything else — which is 2,222 steps before test accuracy reaches 95%. The embedding sparsity climbs monotonically through the plateau, and the weight norm, having grown during memorisation, decays under weight decay during the final cleanup. The three phases — memorisation, circuit formation, cleanup — are explicit, and the sudden jump in test accuracy is the visible tip of a reorganisation that has been underway for thousands of steps.

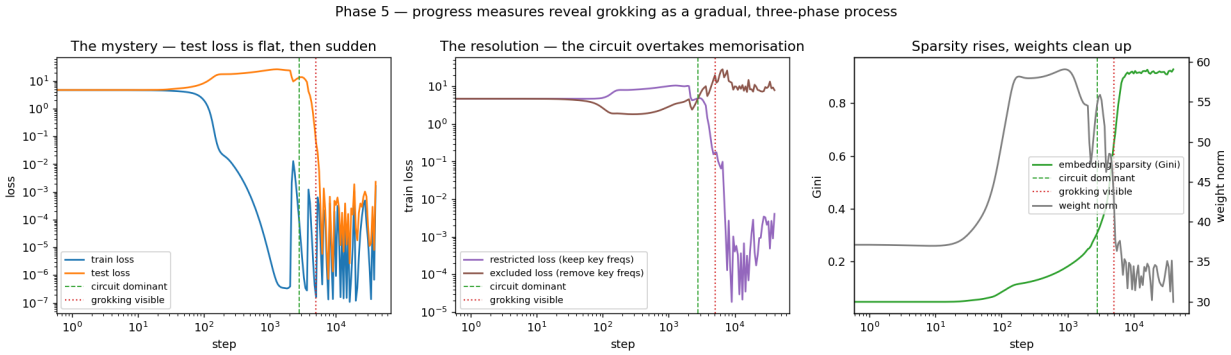

Figure 5: The sudden test-loss cliff (left) hides a gradual process: the circuit overtakes memorisation (middle, crossover) and sparsity climbs (right) well before the loss moves.

What grokking depends on

Varying the training recipe maps the boundaries (Figure 6). Weight decay is essential: with wd = 0 the model memorises and never groks; stronger decay groks sooner (wd = 3/1/0.3 → grok at 1,200 / 4,800 / 18,800 steps). There is a data threshold: a train fraction of 0.2 never groks within budget, while 0.3 / 0.4 / 0.5 grok at 4,800 / 1,200 / 600 steps. And the operation matters: addition, subtraction and multiplication all grok to 100%, but subtraction — though group-isomorphic to addition — groks about five times slower (23,200 vs 4,800 steps), a reminder that isomorphic tasks need not share training dynamics.

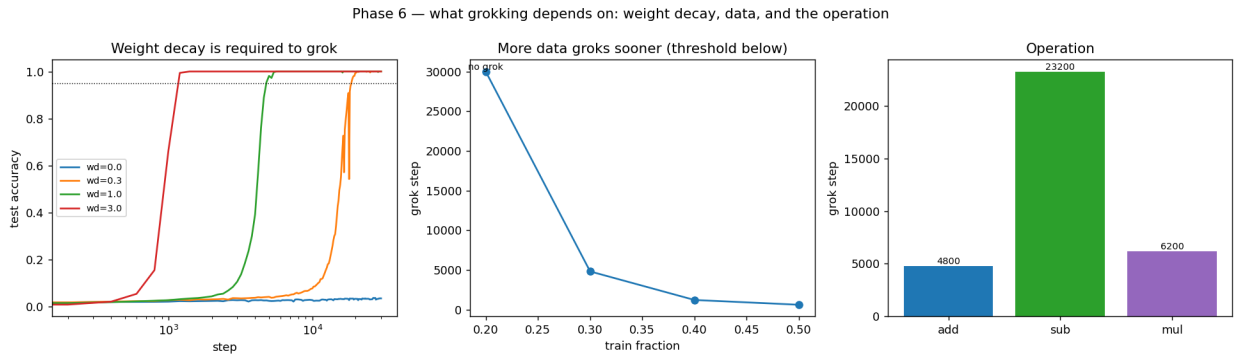

Figure 6: Weight decay is required to grok (left; wd = 0 never groks), a train-fraction threshold exists (middle), and subtraction groks far slower than addition (right).

Generalisation across seeds

A single reverse-engineered run could be a fluke, so we run the analysis on all three seeds (Figure 7). Each independently learns the clock — decision function a function of $(a + b - c)$, fit by key-frequency cosines to $R^2 \geq 0.9998$, peaking at $d = 0$, classifying at 100% — but on a *different* set of key frequencies (seed 0: $\{1, 8, 21\}$; seed 1: $\{1, 18, 45, 56\}$; seed 2: $\{13, 25, 44, 53\}$). The algorithm is universal; the frequencies it selects are seed-dependent.

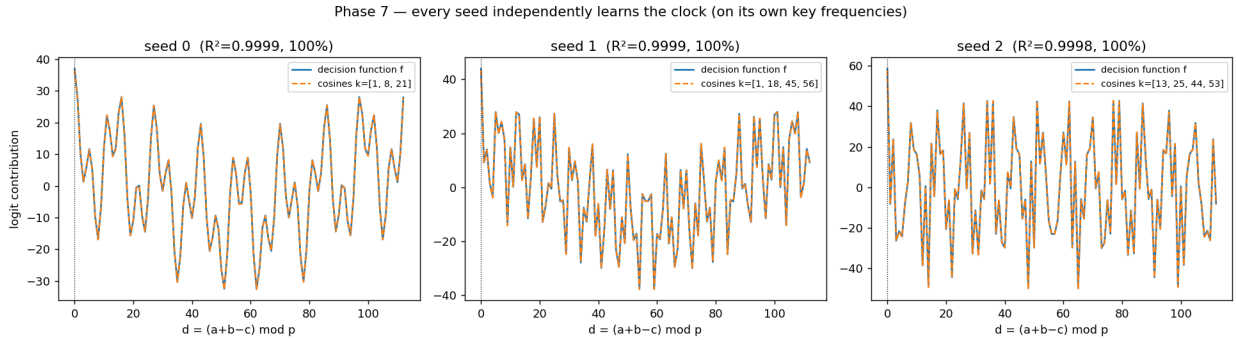

Figure 7: All three independently-trained seeds learn the same clock algorithm, each on its own set of key frequencies.

Scaling with the modulus

To test whether the clock is specific to one prime — and how the circuit's *size* depends on the problem's size — we train the same fixed-capacity model ($d = 128$) to grok on six primes from $p = 53$ to $p = 191$ and measure, for each, the effective number of Fourier frequencies the embedding uses (Figure 8). Every prime groks to $\sim 100\%$, so the clock is not an artefact of $p = 113$. More informatively, the effective number of frequencies stays small — three for the four smaller primes and six for the two largest — across a 3.6× range in p. This is nowhere near proportional to p: a circuit whose size scaled with the problem would use on the order of $p/2$ frequencies (26 to 95 here), not three to six. The count does tick up modestly at the largest primes rather than staying perfectly flat, but the dominant fact is that the circuit size is governed by the model's fixed capacity, not by the modulus — a larger problem is solved with an essentially same-sized Fourier circuit, on

a different set of frequencies. Grokking time itself shows no clean trend with p at these settings, ranging from $\sim$3,600 to $\sim$11,600 steps.

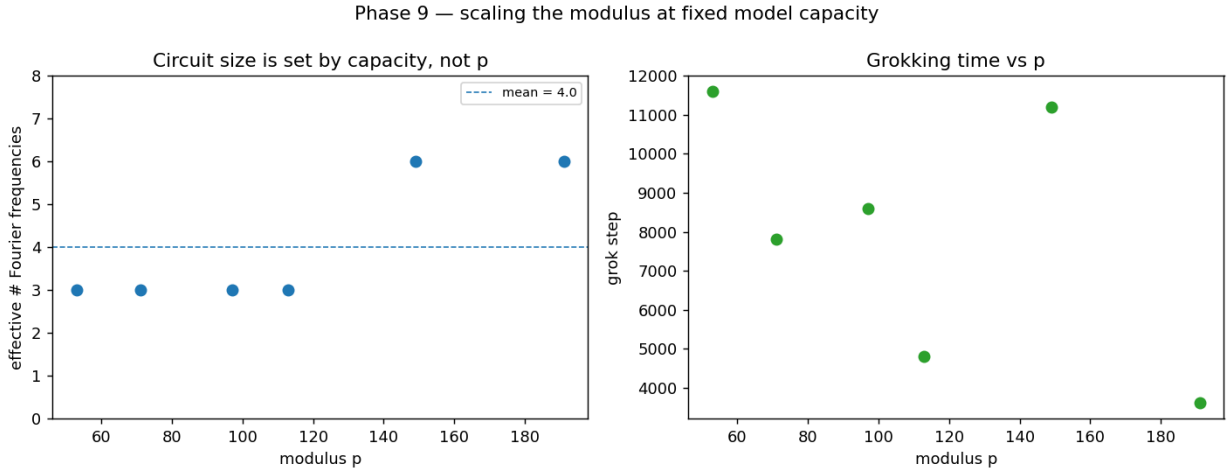

Figure 8: Left: the effective number of Fourier frequencies stays small (3 to 6) across a 3.6x range in p, far from the p/2 a problem-scaled circuit would need. Right: grokking time vs p shows no clean trend.

Limitations and where it breaks

The claims are bounded honestly.

- **Reproduction, not discovery.** The Fourier mechanism, the clock circuit, and the progress-measure framework are due to Nanda et al. (2023), and the clock-vs-pizza distinction to Zhong et al. (2023). Our contribution is a verified from-scratch reproduction with the cross-seed and scaling framings, not a new mechanism.
- **The clock explains 66% of the raw logit variance.** The decision function is a clean closed form and classifies at 100%, but a third of the logit variance is prediction-irrelevant structure we do not model.
- **Clock, not clock-vs-pizza.** We establish the clock but do not run a discriminator that actively rules out the "pizza" algorithm of Zhong et al. (2023); the consistency with their finding is asserted, not tested.
- **No depth ablation.** The ablations vary weight decay, data and operation but not network depth; a two-layer model may learn a messier circuit.
- **Single seed per prime in the scaling study**, and empirical throughout: there are no formal bounds; every error statement is measured.

Related work

Grokking was introduced by Power et al. (2022) on algorithmic datasets. The mechanistic account we reproduce — the Fourier / "clock" circuit and progress measures for grokking — is due to Nanda et al. (2023). Zhong et al. (2023) show that two algorithms ("clock" and "pizza") are possible and that which one a network learns can depend on its architecture. Our from-scratch transformer and Fourier tooling follow the conventions of that line of work, and the causal-ablation and progress-

measure methodology is theirs; the cross-seed universality framing and the fixed-capacity scaling study are our additions.

Conclusion and future work

A one-layer transformer trained on modular addition groks, and the grokked network is a closed-form Fourier algorithm: the embedding is sparse in the Fourier basis, the decision function is a sum of cosines at a few key frequencies ($R^2 = 0.9999$, 100% classification), and those frequencies are causally necessary, sufficient and specific. Progress measures show the circuit forming gradually, thousands of steps before the loss reveals it; grokking requires weight decay and enough data; every seed rediscovers the same algorithm on different frequencies; and at fixed capacity the circuit's size is governed by capacity rather than the modulus. Reproducing these results from scratch, with tested analysis code and full reproducibility, is the contribution.

The natural next steps sharpen the questions this work leaves open. A direct clock-vs-pizza discriminator, applied across architectures (attention-only vs +MLP, one vs two layers), would settle *which* architecture produces *which* algorithm rather than assuming the clock. The multiplication circuit — which lives on the multiplicative group $\mathbb{Z}_p^*$ and should therefore use a discrete-logarithm-indexed basis, since $a \cdot b$ corresponds to adding logarithms — would test whether the same clock reappears on a different group. And the scaling study points at a third: characterising precisely how the learned circuit depends on model capacity.

References

Power, A., Burda, Y., Edwards, H., Babuschkin, I. & Misra, V. (2022). *Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets.* arXiv:2201.02177.

Nanda, N., Chan, L., Lieberum, T., Smith, J. & Steinhardt, J. (2023). *Progress Measures for Grokking via Mechanistic Interpretability.* ICLR.

Zhong, Z., Liu, Z., Tegmark, M. & Andreas, J. (2023). *The Clock and the Pizza: Two Stories in Mechanistic Explanation of Neural Networks.* NeurIPS.